DESENVOLVIMENTO MOBILE · MÓDULO 11 · TRILHA FLUTTER · NÍVEL INTERMEDIÁRIO

# Rede e APIs

HTTP, REST, JSON, pacotes `http` e Dio, interceptadores, erros, tempo real com WebSocket e testes

**Prof. Dr. Francisco dos Santos Viana**

São Luís – Maranhão – Brasil · 2026

# Objetivos do módulo

## Ao final deste módulo você será capaz de

- Explicar requisição, resposta, métodos e códigos de status do HTTP.
- Consumir uma API REST com o pacote `http` e com **Dio**.
- Converter JSON em objetos Dart à mão e com `json_serializable`.
- Usar interceptadores para autenticação, log e renovação de token.
- Tratar falhas de rede com `Result`, tempo limite e retentativas.
- Receber dados em tempo real com WebSocket e testar o código de rede.

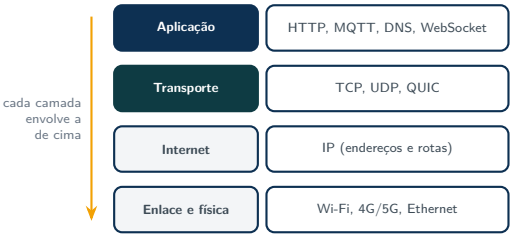
# Roteiro

## Neste módulo

- Fundamentos: redes de computadores
- Fundamentos de HTTP
- O pacote http
- JSON
- Dio
- Robustez e tempo real
- Fechamento

# Fundamentos: redes de computadores

# Redes em camadas: o modelo TCP/IP




### Um pouco de história

Vint Cerf e Bob Kahn publicaram o TCP/IP em 1974; a ARPANET o adotou em 1983. O modelo OSI, de sete camadas, é a referência teórica.

### Protocolo

Conjunto de regras que define o formato das mensagens e a ordem da conversa entre dois computadores.

### Encapsulamento

Cada camada acrescenta seu cabeçalho e entrega à de baixo: a mensagem HTTP vai dentro de um segmento TCP, que vai dentro de um pacote IP, que vai dentro de um quadro de Wi-Fi ou 4G.

### Dica

O app só programa a camada de aplicação; o sistema operacional cuida do resto. Mas conhecer as de baixo explica lentidão e falhas.

# Endereços, portas, NAT e DNS

## Endereço IP e porta

O IP identifica a máquina (IPv4 tem 32 bits; IPv6, 128 bits). A porta identifica o programa: 443 para HTTPS, 8883 para MQTT seguro.

## DNS

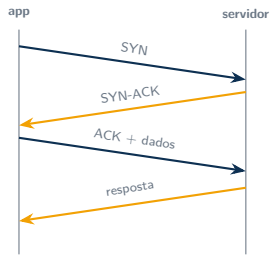
Sistema hierárquico que traduz nomes em IPs: o resolvedor consulta a raiz, depois o domínio de topo (`.dev`) e por fim o servidor responsável. As respostas ficam em cache.




## Por que o servidor não “chama” o celular

O celular fica atrás de um NAT da operadora, com endereço privado: ele abre conexões, mas não recebe conexões novas. Por isso existem as notificações *push* e as conexões mantidas abertas pelo próprio app.

# TCP e UDP




aperto de mão em três vias:
uma ida e volta antes dos dados

| | TCP | UDP |
|---|---|---|
| Conexão | estabelecida antes dos dados | nenhuma: cada pacote é independente |
| Garantias | entrega, ordem e controle de congestionamento | nenhuma: pacotes podem se perder ou chegar fora de ordem |
| Custo | mais idas e voltas | mínimo |
| Usos | HTTP/1.1, HTTP/2, MQTT | DNS, voz, jogos, QUIC |

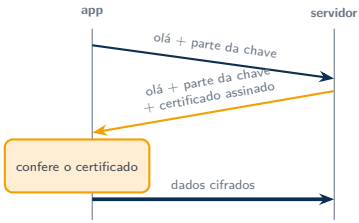
## Latência e largura de banda

**Latência**: tempo de ida e volta de uma mensagem (RTT).
**Largura de banda**: quantos dados passam por segundo. Em apps, as requisições são pequenas: a latência pesa mais.

## Bloqueio na fila

No TCP, um pacote perdido segura todos os seguintes até ser reenviado, mesmo que pertençam a outras requisições.

# TLS: como o HTTPS protege a conversa




TLS 1.3: uma ida e volta para combinar as chaves

## As três garantias do TLS

**Confidencialidade**: ninguém no caminho lê. **Integridade**: ninguém altera sem ser notado. **Autenticidade**: o app fala mesmo com o servidor certo.

## Duas criptografias juntas

A criptografia assimétrica (pares de chaves) é usada só para combinar uma chave secreta; os dados seguem com criptografia simétrica, muito mais rápida (AES ou ChaCha20).

## Cadeia de confiança

O certificado do servidor é assinado por uma autoridade certificadora em que o sistema operacional já confia. Certificado vencido ou de outro domínio: conexão recusada.

# A evolução do HTTP

| Versão | Ano | Novidade | Efeito no app |
|---|---|---|---|
| HTTP/1.0 | 1996 | uma conexão TCP por requisição | lento: aperto de mão a cada pedido |
| HTTP/1.1 | 1997 | conexão reaproveitada (*keep-alive*) | menos apertos de mão, mas um pedido por vez em cada conexão |
| HTTP/2 | 2015 | binário, vários pedidos simultâneos na mesma conexão, cabeçalhos comprimidos | muitas requisições pequenas ficam baratas |
| HTTP/3 | 2022 | roda sobre QUIC (UDP), sem bloqueio na fila do TCP | a conexão sobrevive à troca do Wi-Fi para o 4G |

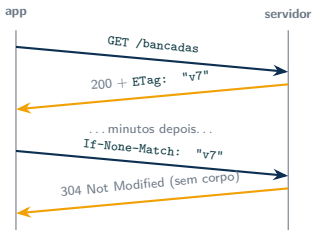
**Um pouco de história**

Tim Berners-Lee criou o HTTP e a Web no CERN, entre 1989 e 1991. Três décadas depois, o mesmo modelo de requisição e resposta move quase todos os apps.

**Quem fala o quê**

OkHttp usa HTTP/2; o Cronet, motor de rede do Chrome, também HTTP/3. No Flutter, `cronet_http` e `cupertino_http` usam as pilhas nativas do Android e do iOS.

# Cache HTTP e requisições condicionais




**ETag**

Identificador da versão de um recurso. Na próxima vez, o app pergunta "mudou desde a v7?"; se não mudou, a resposta 304 vem vazia e o app usa a cópia que já tem.

| `Cache-Control` | **Significado** |
|---|---|
| `max-age=60` | pode reutilizar por 60 s sem perguntar |
| `no-cache` | pode guardar, mas pergunte antes de usar |
| `no-store` | não guarde (dados sensíveis) |

**Dica**

OkHttp e Dio têm cache HTTP: configurá-lo economiza dados móveis e bateria.

# Serialização: de objetos a bytes

**Serialização**

Converter objetos em bytes para enviar ou guardar; desserializar é o caminho de volta. Respostas comprimidas (gzip, brotli) reduzem muito o JSON.

| Formato | Características | Uso típico |
|---|---|---|
| JSON | texto legível, sem esquema obrigatório | APIs REST |
| Protocol Buffers | binário, com esquema, menor e mais rápido | gRPC, dados volumosos |
| CBOR, MessagePack | JSON em formato binário | IoT |

**Evolução sem quebrar**

Apps antigos continuam instalados por meses. O servidor só acrescenta campos; o app ignora campos desconhecidos e tem valores padrão para os que faltarem. Mudanças incompatíveis vão para uma nova versão da API (`/v2`).

**JSON grande trava a tela**

Decodificar megabytes de JSON é trabalho de CPU: faça fora da *thread* da interface (Módulo 06).

# Redes móveis: latência, energia e falhas

## O rádio gasta mesmo parado

Depois de cada transferência, o rádio celular fica alguns segundos em modo de alta potência antes de voltar ao repouso. Várias requisições espalhadas gastam mais bateria que as mesmas requisições agrupadas.

| Método | Repetir é seguro? |
|---|---|
| `GET`, `PUT`, `DELETE` | sim: são idempotentes (repetir dá o mesmo resultado) |
| `POST` | não: repetir pode criar duas reservas |

## Defesas contra falhas

**Tempo limite** em toda requisição; **novas tentativas** com espera exponencial e um pouco de aleatoriedade (*jitter*); **disjuntor** que para de insistir num servidor fora do ar.

### "Conectado" não é "com internet"

Wi-Fi de hotel com página de login, rede fraca ou servidor fora do ar: a única prova de rede é a requisição que funcionou.

# Tempo real: *polling*, SSE, WebSocket e *push*

| Técnica | Como funciona | Custo | Quando usar |
|---|---|---|---|
| *Polling* | o app pergunta de tempos em tempos | alto: muitas requisições vazias | dados que mudam pouco |
| *Long polling* | o servidor segura a resposta até haver novidade | médio | quando nada melhor está disponível |
| SSE | o servidor envia eventos por uma resposta HTTP que não termina | baixo; só servidor → app | painéis e notícias ao vivo |
| WebSocket | canal aberto nos dois sentidos | baixo por mensagem; conexão aberta | chat, leituras ao vivo |
| MQTT | publicação e assinatura por tópicos | muito baixo | sensores e IoT (Módulo 13) |
| *Push* (FCM, APNs) | o sistema entrega mesmo com o app fechado | quase zero para o app | avisos e alertas (Módulo 12) |

**Uma conexão só, do sistema**

O *push* usa uma única conexão mantida pelo sistema para todos os apps: é o jeito mais econômico de receber avisos.

**Atenção**

Conexões abertas não sobrevivem ao segundo plano: o sistema as encerra. Para avisar um app fechado, só o *push*.

# Fundamentos de HTTP

# Requisição e resposta




**Partes de uma requisição**

**Método** + **URL** + **cabeçalhos** + **corpo** (opcional). A resposta traz **código de status**, cabeçalhos e corpo, em geral JSON.

| Método | Uso em REST |
|---|---|
| `GET` | ler um recurso ou uma lista |
| `POST` | criar um recurso |
| `PUT` | substituir um recurso inteiro |
| `PATCH` | alterar parte de um recurso |
| `DELETE` | remover um recurso |

| Status | Significado |
|---|---|
| `2xx` | sucesso (200 OK, 201 Created, 204) |
| `4xx` | erro do cliente (400, 401, 403, 404, 409, 422) |
| `5xx` | erro do servidor (500, 502, 503) |

# Estilos de API

| Estilo | Como funciona | Quando usar |
|---|---|---|
| **REST** | recursos em URLs, métodos HTTP, JSON | padrão da maioria dos backends; foco deste módulo |
| **GraphQL** | uma URL; o cliente descreve os campos que quer | telas que combinam muitos dados; evita várias chamadas |
| **gRPC** | contratos Protocol Buffers sobre HTTP/2 | comunicação entre serviços; alto desempenho |
| **WebSocket** | conexão aberta nos dois sentidos | tempo real: leituras ao vivo, chat |
| **MQTT** | publicação e assinatura por tópicos | IoT com rede instável (Módulo 13) |

**Explore antes de codificar**

Teste a API no **Bruno**, **Postman** ou `curl`. Se ela publica um documento **OpenAPI**, leia os contratos ali.

**Atenção**

Use sempre **HTTPS**. O Android bloqueia HTTP sem TLS por padrão e o iOS exige TLS pelo ATS.

# O pacote http

# Preparando o projeto

```
$ flutter pub add http
```

**android/app/src/main/AndroidManifest.xml**

```
<manifest ...>
  <uses-permission
      android:name="android.permission.INTERNET" />
  <application ...>
```

**macos/Runner/*.entitlements**

```
<key>com.apple.security.network.client</key>
<true/>
```

**Funciona no debug, falha na loja**

O modo debug do Android já inclui a permissão `INTERNET`; a versão *release* não. Sem a linha no manifesto, o app publicado não acessa a rede.

**Plataformas**

iOS e web não pedem permissão para rede. No macOS, o app roda em *sandbox* e precisa do *entitlement* de cliente de rede.

# Um serviço de API com `http`

**data/services/api_service.dart**

```dart
class ApiService {
  ApiService({http.Client? client})
      : _c = client ?? http.Client();
  final http.Client _c;
  static const _base = 'https://api.labmonitor.dev/v1';

  Future<List<BancadaDto>> getBancadas() async {
    final r = await _c.get(Uri.parse('$_base/bancadas'))
        .timeout(const Duration(seconds: 10));
    if (r.statusCode != 200) throw ApiException(r.statusCode);
    final lista = jsonDecode(r.body) as List<dynamic>;
    return lista.map((j) => BancadaDto.fromJson(j)).toList();
  }
  Future<void> reservar(int id, DateTime ate) async {
    final r = await _c.post(
      Uri.parse('$_base/bancadas/$id/reservas'),
      headers: {'Content-Type': 'application/json'},
      body: jsonEncode({'ate': ate.toIso8601String()}));
    if (r.statusCode != 201) throw ApiException(r.statusCode);
  }
}
```

## Boas práticas

- Um `http.Client` reutilizado mantém as conexões abertas.
- Receber o cliente no construtor permite testar com `MockClient`.
- Sempre defina um tempo limite.
- Confira o status antes de ler o corpo.

### Dica

`ApiException` é uma classe sua; não use `HttpException` de `dart:io`, que não existe na web.

# JSON

# JSON à mão com *patterns*

domain/dto/bancada_dto.dart

```dart
class BancadaDto {
  BancadaDto(this.id, this.nome, this.reservadaAte);
  final int id;
  final String nome;
  final DateTime? reservadaAte;

  factory BancadaDto.fromJson(Object? json) {
    if (json case {
      'id': int id,
      'nome': String nome,
      'reservada_ate': String? ate,
    }) {
      return BancadaDto(id, nome,
          ate == null ? null : DateTime.parse(ate));
    }
    throw FormatException('Bancada inválida: $json');
  }

  Map<String, Object?> toJson() => {'id': id, 'nome': nome,
      'reservada_ate': reservadaAte?.toIso8601String()};
}
```

### *Pattern* de mapa

O `if case` confere a estrutura e os tipos e extrai os campos em uma só expressão (Dart 3, Módulo 02). JSON inesperado vira um erro claro, não um `null` silencioso.

### Atenção

`jsonDecode` devolve `dynamic`. Acessar `json['nome']` sem conferir o tipo adia o erro para longe da sua causa.

# JSON com geração de código

**Com json_serializable**

```dart
import 'package:json_annotation/json_annotation.dart';
part 'bancada_dto.g.dart';
@JsonSerializable(fieldRename: FieldRename.snake)
class BancadaDto {
  BancadaDto({required this.id, required this.nome,
      this.reservadaAte});
  final int id;
  final String nome;
  final DateTime? reservadaAte;    // "reservada_ate"
  factory BancadaDto.fromJson(Map<String, dynamic> json) =>
      _$BancadaDtoFromJson(json);
  Map<String, dynamic> toJson() => _$BancadaDtoToJson(this);
}
```

```
$ flutter pub add json_annotation
$ flutter pub add -d json_serializable build_runner
$ dart run build_runner watch
```

## Quando gerar

Com poucos modelos, o código à mão basta. Com dezenas de modelos e campos aninhados, a geração evita erros de digitação.

### `freezed`

Gera também `copyWith`, igualdade, `toString` e uniões seladas; muito usado com Riverpod e Bloc.

### JSON grande

Decodifique fora da *thread* da UI com `Isolate.run` (Módulo 06).

# Dio

# Configurando o Dio

**data/services/cliente_http.dart**

```dart
Dio criarDio(TokenStorage tokens) {
  final dio = Dio(BaseOptions(
    baseUrl: 'https://api.labmonitor.dev/v1/',
    connectTimeout: const Duration(seconds: 5),
    receiveTimeout: const Duration(seconds: 10),
  ));
  dio.interceptors.addAll([
    InterceptorsWrapper(onRequest: (opcoes, handler) async {
      final t = await tokens.lerAcesso();
      if (t != null) opcoes.headers['Authorization'] = 'Bearer $t';
      handler.next(opcoes);
    }),
    if (kDebugMode) LogInterceptor(responseBody: true),
  ]);
  return dio;
}
// uso: o JSON já vem decodificado
final r = await dio.get<List<dynamic>>('bancadas',
    queryParameters: {'lab': 3, 'livres': true});
final bancadas =
    r.data!.map((j) => BancadaDto.fromJson(j)).toList();
```

## Dio

Cliente HTTP com URL base, tempos limite, **interceptadores**, cancelamento, *upload* com progresso e decodificação automática de JSON.

## Interceptador

Código que roda em toda requisição, resposta ou erro: incluir token, registrar log, medir tempo, renovar sessão.

# Erros e renovação de token

**Traduzindo DioException**

```
String mensagemDe(DioException e) => switch (e.type) {
  DioExceptionType.connectionTimeout ||
  DioExceptionType.receiveTimeout =>
      'O servidor demorou a responder.',
  DioExceptionType.connectionError =>
      'Sem conexão com a internet.',
  DioExceptionType.badResponse => switch (
      e.response?.statusCode) {
    401 => 'Sessão expirada.',
    404 => 'Recurso não encontrado.',
    _ => 'Erro no servidor.',
  },
  _ => 'Falha inesperada.',
};
```

**Renovando o token após 401**

```
QueuedInterceptorsWrapper(
  onError: (e, handler) async {
    if (e.response?.statusCode != 401) {
      return handler.next(e);
    }
    final ok = await auth.renovarToken();
    if (!ok) return handler.next(e);
    final repetida = await dio.fetch(
        e.requestOptions);
    handler.resolve(repetida);
  },
)
```

**Dica**

A versão *Queued* processa um erro por vez: várias requisições com 401 disparam uma única renovação.

# Upload, download e cancelamento

**Foto do equipamento com progresso**

```dart
final cancelar = CancelToken();

Future<void> enviarFoto(int id, String caminho) =>
    dio.post('equipamentos/$id/foto',
      data: FormData.fromMap({
        'arquivo': await MultipartFile.fromFile(caminho,
            filename: 'foto.jpg'),
      }),
      cancelToken: cancelar,
      onSendProgress: (enviado, total) =>
          progresso.value = enviado / total,
    );

Future<void> baixarRelatorio(String destino) =>
    dio.download('relatorios/mensal.pdf', destino,
        cancelToken: cancelar);

// ao sair da tela
@override
void dispose() { cancelar.cancel(); super.dispose(); }
```

**`multipart/form-data`**

Formato usado para enviar arquivos junto com campos de formulário.

**Dica**

Cancele requisições quando o usuário sai da tela: economiza dados e evita atualizar um estado que já não existe.

**Atenção**

`MultipartFile.fromFile` usa `dart:io`; na web, use `MultipartFile.fromBytes`.

# Robustez e tempo real

# Falhas de rede são normais

**utils/retentativa.dart**

```dart
Future<T> comRetentativa<T>(Future<T> Function() acao,
    {int maximo = 3}) async {
  for (var i = 0; ; i++) {
    try {
      return await acao();
    } on DioException catch (e) {
      final transitoria =
          e.type != DioExceptionType.badResponse ||
          (e.response?.statusCode ?? 0) >= 500;
      if (!transitoria || i + 1 >= maximo) rethrow;
      // 0,5 s, 1 s, 2 s ...
      await Future.delayed(
          Duration(milliseconds: 500 * (1 << i)));
    }
  }
}
```

## Estratégia

- Repita só erros **transitórios** (tempo limite, 5xx), nunca 400 ou 401.
- Espere cada vez mais (*backoff* exponencial).
- O repositório converte a exceção em `Result` (Módulo 08).
- Mostre dados do banco local enquanto a rede falha (Módulo 10).

**Dica**

`connectivity_plus` informa o tipo de rede (Wi-Fi, móvel), não se há internet de fato.

# Tempo real com WebSocket

**Leituras ao vivo da bancada**

```dart
import 'package:web_socket_channel/web_socket_channel.dart';

class LeiturasAoVivo {
  WebSocketChannel? _canal;

  Future<Stream<Leitura>> conectar(int bancada) async {
    final canal = WebSocketChannel.connect(
        Uri.parse('wss://api.labmonitor.dev/ws'));
    await canal.ready;                    // conexão aberta
    canal.sink.add(jsonEncode({'assinar': bancada}));
    _canal = canal;
    return canal.stream
        .map((m) => Leitura.fromJson(jsonDecode(m)));
  }

  Future<void> fechar() async => _canal?.sink.close();
}
```

**WebSocket**

Conexão que fica aberta: o servidor envia dados assim que eles existem, sem o app perguntar de tempos em tempos (*polling*).

**Dica**

O resultado é um `Stream`: use `StreamBuilder` ou converta no view model (Módulo 06).

**Atenção**

Conexões caem. Reconecte com *backoff* e feche o canal quando a tela sai.

# Testando código de rede

test/api_service_test.dart

```dart
import 'package:http/http.dart' as http;
import 'package:http/testing.dart';

void main() {
  test('getBancadas converte o JSON', () async {
    final cliente = MockClient((req) async {
      expect(req.url.path, '/v1/bancadas');
      return http.Response(
          '[{"id":1,"nome":"B1","reservada_ate":null}]', 200);
    });
    final api = ApiService(client: cliente);
    final lista = await api.getBancadas();
    expect(lista.single.nome, 'B1');
  });

  test('status 500 lança ApiException', () async {
    final api = ApiService(client: MockClient(
        (_) async => http.Response('', 500)));
    expect(api.getBancadas(), throwsA(isA<ApiException>()));
  });
}
```

## Sem rede nos testes

`MockClient` responde o que você quiser, de forma instantânea e determinística. Teste sucesso, erro, JSON malformado e tempo limite.

**Dica**

Com Dio, substitua o `httpClientAdapter` ou use o pacote `http_mock_adapter`.

# Segurança e escolha da biblioteca

**Segurança na rede**

- Só HTTPS; nada de desativar a validação de certificado.
- **Nunca** coloque segredos no app: o binário pode ser extraído. Chaves privadas ficam no backend.
- Tokens em armazenamento seguro (Módulo 10).
- Em apps sensíveis, considere *certificate pinning*.

| Pacote | Indicado para |
|---|---|
| `http` | simples, oficial do time Dart; poucas chamadas |
| **Dio** | interceptadores, *upload*, cancelamento; apps reais |
| `retrofit` | interface anotada gerando cliente Dio (como o Retrofit do Kotlin) |
| `web_socket_channel` | tempo real |

**Dica**

O **DevTools** tem uma aba *Network* que mostra cada requisição do app em execução.

# Fechamento

# Resumo

## Resumo

- HTTP: método, URL, cabeçalhos e corpo; a resposta traz status e dados.
- `http` é simples; Dio acrescenta interceptadores, cancelamento e progresso.
- Valide o JSON com *patterns* ou gere o código com `json_serializable`.
- Trate erros por tipo, repita só falhas transitórias e devolva `Result`.
- WebSocket para tempo real; `MockClient` para testes sem rede.

# Glossário (1/2)

| Termo | Significado |
|---|---|
| **Protocolo** | regras de formato e de ordem das mensagens entre computadores |
| **TCP/IP** | pilha de protocolos da internet: aplicação, transporte, internet e enlace |
| **DNS** | sistema que traduz nomes de domínio em endereços IP |
| **NAT** | tradução de endereços que esconde o celular atrás de um IP público |
| **TCP** | transporte com conexão, entrega garantida e ordem |
| **UDP** | transporte sem conexão nem garantias, de baixa latência |
| **TLS** | protocolo que cifra e autentica a conexão (o “S” do HTTPS) |
| **Latência (RTT)** | tempo de ida e volta de uma mensagem |
| **HTTP/2 e HTTP/3** | vários pedidos na mesma conexão; o HTTP/3 roda sobre QUIC (UDP) |

# Glossário (2/2)

| Termo | Significado |
|---|---|
| **REST** | estilo de API com recursos em URLs e métodos HTTP |
| ***ETag*** | identificador de versão que permite a resposta 304 sem corpo |
| **Serialização** | conversão de objetos em bytes (JSON, Protocol Buffers) |
| **Idempotência** | repetir a operação produz o mesmo resultado |
| **`http` e Dio** | pacotes de cliente HTTP do Dart; o Dio tem interceptadores e cancelamento |
| **`json_serializable`** | gera o código de conversão entre JSON e classes Dart |
| **Interceptador** | código que examina ou altera toda requisição e resposta |
| **WebSocket** | canal aberto nos dois sentidos para tempo real |

# Questões de revisão

1. Descreva, camada por camada, o que acontece quando o app chama `GET https://api.labmonitor.dev/v1/bancadas`.
2. Por que o servidor não consegue abrir uma conexão até o celular? Que soluções existem?
3. Por que, em apps, a latência costuma pesar mais que a largura de banda?
4. Que três garantias o TLS oferece e como o app sabe que fala com o servidor certo?
5. O que o HTTP/3 traz de especialmente útil para um celular em movimento?
6. Explique como *ETag* e a resposta 304 economizam dados.
7. Que cuidados permitem mudar a API sem quebrar versões antigas do app?
8. Uma requisição `POST /reservas` deu tempo limite esgotado. É seguro repetir? Como tornar seguro?

# Exercícios

**Exercício 1 — API pública**

Consuma uma API pública (por exemplo, a de CEP ou de clima) com `http` e mostre carregando, erro e dados.

**Exercício 2 — Dio**

Migre o serviço para Dio com interceptador de token e mensagens de erro traduzidas.

**Exercício 3 — Testes**

Escreva testes com `MockClient` para 200, 404, 500 e JSON malformado.

**LabMonitor — Etapa 11**

Sincronize bancadas e leituras com a API em Dio, salve no Drift (Módulo 10) e mostre as leituras ao vivo via WebSocket.

# Para aprofundar

- Rede no Flutter: `https://docs.flutter.dev/data-and-backend/networking`
- JSON e serialização: `https://docs.flutter.dev/data-and-backend/serialization/json`
- Pacote http: `https://pub.dev/packages/http` Dio: `https://pub.dev/packages/dio`
- json_serializable: `https://pub.dev/packages/json_serializable`
- WebSocket: `https://docs.flutter.dev/cookbook/networking/web-sockets`

# Respostas das questões de revisão

1. O DNS traduz o nome em IP; o TCP abre a conexão na porta 443; o TLS combina as chaves e confere o certificado; o HTTP envia a requisição cifrada; tudo viaja em pacotes IP por Wi-Fi ou rede móvel.
2. O celular fica atrás de NAT, com endereço privado, e não aceita conexões novas. Soluções: notificações *push* (FCM, APNs) ou conexões abertas pelo próprio app (WebSocket, MQTT).
3. As requisições são pequenas: o tempo de ida e volta (aperto de mão, DNS, TLS) domina o tempo de transferência.
4. Confidencialidade, integridade e autenticidade. O certificado do servidor é assinado por uma autoridade em que o sistema confia e precisa valer para aquele domínio.
5. Roda sobre QUIC: sem bloqueio na fila do TCP e com conexão que sobrevive à troca de rede, como do Wi-Fi para o 4G.
6. O app envia a versão que tem (`If-None-Match`); se nada mudou, o servidor responde 304 sem corpo e o app reutiliza a cópia local.
7. Só acrescentar campos, ignorar campos desconhecidos (o padrão do `json_serializable`), ter valores padrão e criar `/v2` para mudanças incompatíveis.
8. Não, pois `POST` não é idempotente e pode criar duas reservas. Envie uma chave de idempotência, para o servidor reconhecer a repetição.

# Fim do Módulo 11

**Próximo:** Módulo 12 — Firebase com FlutterFire: autenticação, Firestore, Storage, mensagens e Crashlytics.